

7/21 - 6조 transformer 실습

Transformer 기반 미니 언어모델 Baseline 개선 실험 보고서

조원

- 김광현
- 이민형
- 임채현
- 임유리

1. 실습 목적

본 실습에서는 Transformer 기반 미니 언어모델의 구조와 학습 과정을 이해하고, 주어진 Baseline 모델의 설정을 변경하여 성능 변화를 비교하는 것을 목적으로 하였다.

제시된 개선 후보는 다음과 같다.

1. 모델 크기(`n_layer` , `n_head` , `n_embd`)
2. 학습 반복 수(`MAX_ITERS`)
3. 문맥 길이(`block_size`)
4. 생성 파라미터(`temperature` , `top_k`)
5. 데이터 양
6. Causal Mask 유무

제한된 실습 시간을 고려하여 본 조는 다음 두 가지 요소를 선택하였다.

- 모델 크기 변경
- 학습 반복 수 변경

각 실험에서는 비교 대상 이외의 조건을 동일하게 유지하고, Loss 곡선과 생성 문장을 통해 설정 변화가 모델 성능에 미치는 영향을 분석하였다.

2. 데이터 및 실험 환경

2.1 데이터

- 제공된 김유정 작품 텍스트 데이터
- 문자 단위 Character-level Tokenization 사용
- 전체 텍스트에서 등장하는 각 문자를 하나의 토큰으로 처리

본 실험에서는 별도의 외부 데이터를 추가하지 않고 동일한 데이터셋을 모든 모델에 사용하였다.

2.2 실험 환경

- Python
- PyTorch
- Google Colab
- Transformer Decoder 기반 미니 언어모델
- Next Token Prediction 방식
- Causal Mask 적용

3. 모델의 기본 구조

본 모델은 입력된 문자 시퀀스를 바탕으로 다음 문자를 예측하는 자기회귀 언어모델이다.

전체 처리 과정은 다음과 같다.

```
입력 텍스트
→ 문자 단위 토큰화
→ Token Embedding
→ Position Embedding
→ Masked Self-Attention
→ Feed Forward Network
→ Linear Layer
→ 다음 문자 확률 예측
```

3.1 Character-level Tokenization

텍스트를 단어나 형태소가 아닌 문자 단위로 분리한다.

예를 들어 다음 문장은

오늘도

다음과 같이 처리될 수 있다.

오 / 늘 / 도

Character-level Tokenizer는 구현이 단순하고 사전에 없는 단어도 처리할 수 있다는 장점이 있다. 반면 단어 단위의 의미를 직접 표현하지 못하고, 긴 문맥을 학습하기 위해 더 많은 토큰이 필요하다는 한계가 있다.

3.2 Embedding

각 문자 토큰은 정수 ID로 변환된 후 `n_embd` 차원의 벡터로 변환된다.

- Token Embedding: 각 문자의 의미적 표현
- Position Embedding: 문자가 등장한 위치 정보

Transformer는 입력 순서를 자동으로 인식하지 못하기 때문에 Position Embedding이 필요하다.

3.3 Self-Attention

Self-Attention은 현재 토큰을 처리할 때 이전 토큰 중 어떤 토큰을 중요하게 참고할지를 계산한다.

각 토큰은 다음 세 벡터로 변환된다.

- Query
- Key
- Value

Query와 Key의 유사도를 이용해 Attention Score를 계산하고, 이를 바탕으로 Value를 가중합한다.

3.4 Causal Mask

본 모델은 다음 문자를 예측하는 자기회귀 모델이므로 미래 토큰을 미리 볼 수 없어야 한다.

따라서 Causal Mask를 적용하여 각 토큰이 현재 위치보다 뒤에 있는 토큰을 참고하지 못하도록 제한하였다.

첫 번째 토큰 → 첫 번째 토큰만 참고
두 번째 토큰 → 첫 번째, 두 번째 토큰 참고
세 번째 토큰 → 첫 번째, 두 번째, 세 번째 토큰 참고

4. 실험 설계

본 조는 모델 크기와 학습 반복 수를 각각 변경하였다.

4.1 공통 설정

모든 실험에서 다음 조건은 동일하게 유지하였다.

항목	설정
Tokenizer	Character-level
<code>block_size</code>	128
데이터셋	동일한 김유정 작품 텍스트
Causal Mask	적용
생성 길이	동일하게 설정
시작 프롬프트	동일하게 설정
<code>temperature</code>	동일하게 설정
<code>top_k</code>	동일하게 설정

생성 문장을 비교할 때 모델 설정 이외의 영향을 줄이기 위해 같은 시작 프롬프트와 같은 생성 파라미터를 사용하였다.

5. 실험 1 — 모델 크기 변경

5.1 무엇을 변경했는가?

모델의 크기를 결정하는 다음 세 가지 값을 변경하였다.

- `n_layer`: Transformer Block 수
- `n_head`: Multi-Head Attention의 Head 수
- `n_embd`: 토큰 Embedding 차원

세 모델 모두 문맥 길이는 `block_size = 128` 로 동일하게 유지하였다.

모델	<code>block_size</code>	<code>n_layer</code>	<code>n_head</code>	<code>n_embd</code>
Small	128	3	3	96
Baseline	128	6	6	192
Large	128	12	12	384

모델 크기 실험에서는 학습 반복 수를 동일하게 설정하여 모델 구조의 변화만 비교해야 한다.

모델 크기 비교 실험의 공통 MAX_ITERS: 100

5.2 예상 결과

모델 크기가 증가하면 학습 가능한 파라미터 수와 표현력이 증가한다.

따라서 Small 모델보다 Baseline 모델과 Large 모델이 문자 간 관계와 문맥 패턴을 더 복잡하게 학습할 수 있을 것으로 예상하였다.

예상 결과는 다음과 같다.

- Small 모델은 학습 속도가 빠르지만 표현력이 제한적일 것이다.
- Baseline 모델은 학습 비용과 성능 사이에서 중간 수준을 보일 것이다.
- Large 모델은 가장 낮은 Loss와 가장 자연스러운 문장을 생성할 가능성이 있다.
- 모델이 커질수록 학습 시간과 메모리 사용량이 증가할 것이다.
- 데이터 양과 학습 횟수가 충분하지 않으면 Large 모델의 성능 향상이 크지 않을 수 있다.

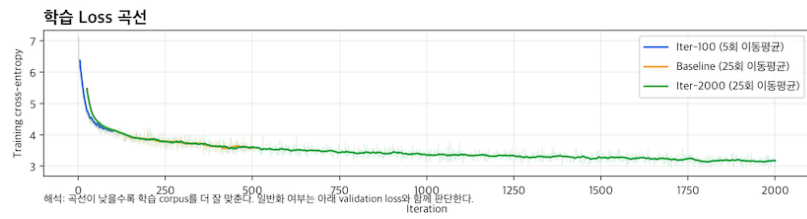
5.3 실제 결과

5.3.1 최종 Loss

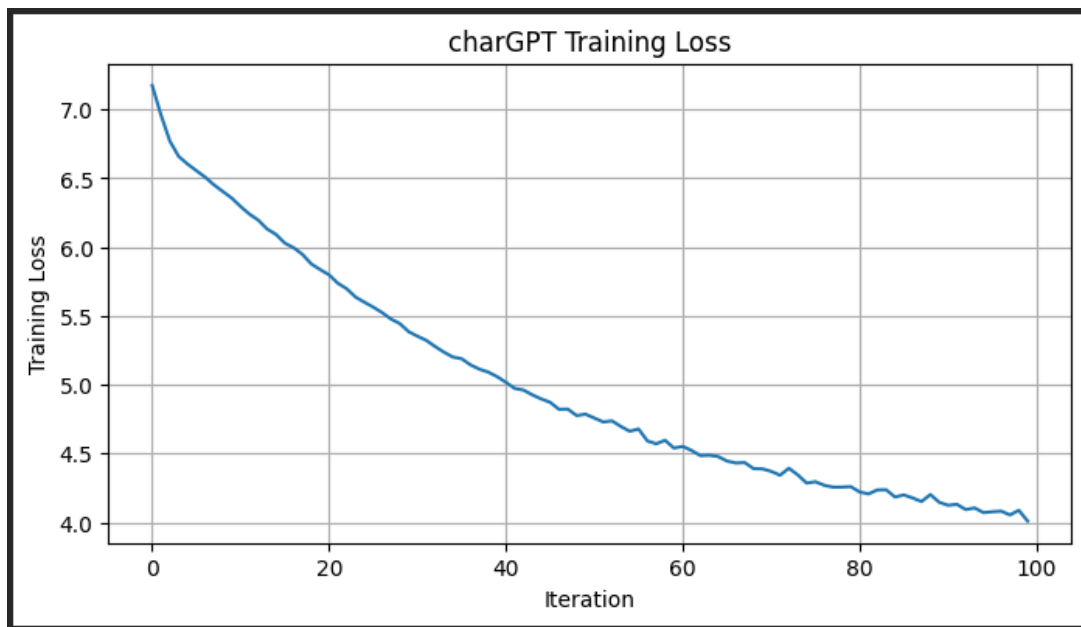
모델	Loss	학습 시간
Small	4.00911	3.8초
Baseline	3.68497	17.4초
Large	3.65684	52.2초

5.3.2 Loss 곡선

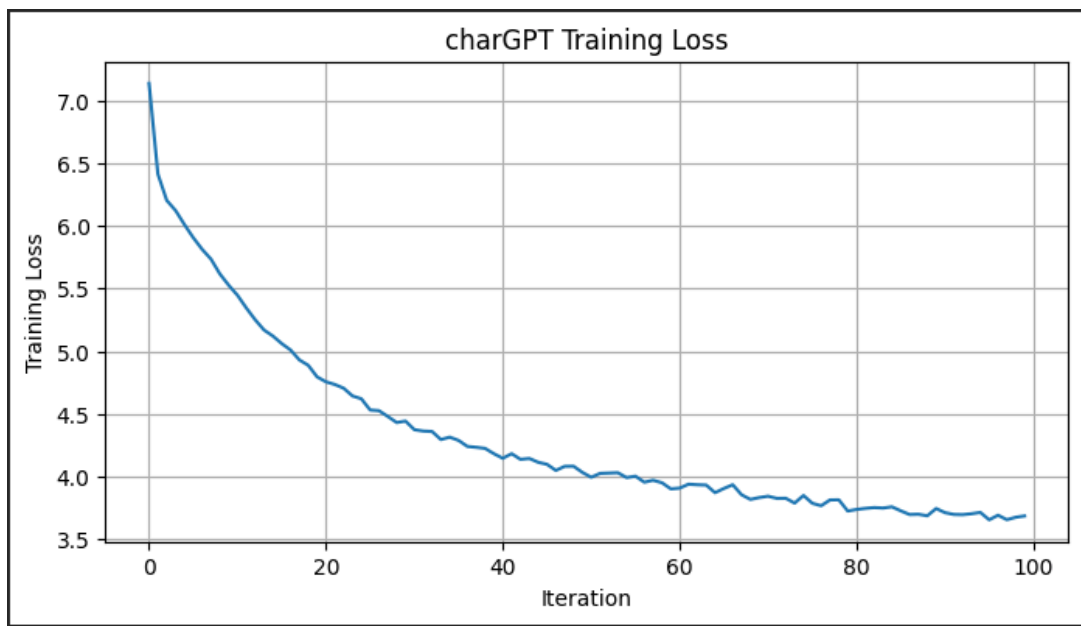
아래에 모델 크기별 Loss 곡선을 삽입한다.



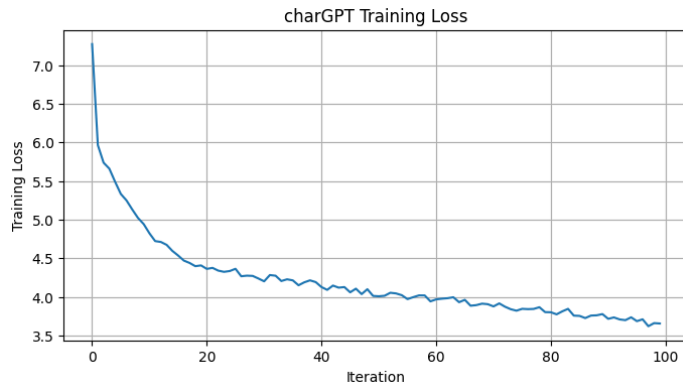
Small



Baseline



Large



그래프의 가로축은 학습 반복 횟수이며, 세로축은 Loss이다.

5.3.3 생성 문장 비교

모든 모델에 동일한 시작 프롬프트, 생성 길이, `temperature`, `top_k` 를 적용한다.

학습 전 생성 결과

오늘도 또 우리 수탉이 막 꽃달피는걸로헬나용밤인혁찬겨건뽕넛논쫓육넛쌈넛랏힐뽕짓곤덜용힐!매니택떡점텃탕합뜰!짓!.텃엄뽕짚흥툏
커풍격각길광당칭취된남민텃촉해푸벌논뽕게년편회출간찌닐않돌감름낭논툏랍헐랄짚터죽해동흙덜날민밀체역쌍편흙뽕찌않당둔럽맘민짜맘
곳쌈합남곰뽕뽕쌈합편고뽕덜호합죽곶앗뽕쌈맘랏찌낙공넛쌈거짚쌈합뽕흥뽕낙공알쌈곤남회갈합합남섞곰넛남웁인맘불희곰낙섞뽕두값
털합날웁합짚랏앗홀티넛날문커착합합섞뽕남뽕뽕넛넛조짚쌈합곰넛남섞뽕쌈넛랏앗흥남맘나랭곰낙동뽕회합곰맘쌍외뽕공넛호나남곰넛
남합합남춤인흥맘싼반외불뽕씩황커러쌈흥뽕호낙공툏롱뽕넛날딘합침맘낙초짚합곰건감맘뽕쌍편흙나조툏낙업곰알산낙곰방흥합노뽕낙곰방낙
해곰

Baseline 모델

오늘도 또 우리 수탉이 막 꽃으면 나 아내리 내를 것이 한다. 한 말고 그러나 사리가 그 말 그 없는 하였다가 하고 그러니다. 한
그리 그 것이 이 하게까 이 이 아내리가 그러나서 그 말이 나서서 그러라 다. 나무 말 아키 나보고 하고 그러나 그렇지 그래 한
아 나는 그는 아니다 이 말 가 가?"
" 아서, 것이는 이다. 내의 아내리나라는 나 그 그러나서 아니," 말이는 이 한 그리다. 이 내 아니 말 아니 아니 한 내려
지, 것이 그리들 그리다.
"그런 것이가 나,
"그리 그 내려 그런 내리고,
"아나, 가 아 그는 말을 그래!"
아키지."
"
"아내는 그는 하고 이 것 것이라 아내리니다 사람 말로 그 그 내지는다. 그럼 말이 것어? 하고. 말이나 이 사람의 말이 그 그는
가 아키어?"
"이 가 아니 그러 아내는 그래 그렇게 이 말로 그 것이 하면 그래 내 가 이 나 내려 가 내리를 하는 나서 내는 것이 하고 아서
그런 그러 이 나서는 한 그 하고 그 나는 한다. 이고 그리며 그러 이는 말이 아내 나도 나

Small 모델

오늘도 또 우리 수탉이 막 쫓이 아, 사다. 들 하고 한 사고 아 아내를 하 아하였 그렇다다. 것이 나 그러 사다 아서 그러 아내
하는 없는 하고 그렇고 그는 아 나 그 그리는 한, 하 하고 이 이 하 아 아 그 한 한 하지 한 하 아서 하는 이가 하 것에 것도
하였다 그 내는 사. 나 이 아서 한이다 아이 아 그리 그 나 사로 그 이다시 그러다. 아지지. 한
" 것이 하다 그러
"하 그 그리고 나 아니 아다. 내이 그 그 하다.
아서 그 한 이 없고 것을 하는 하다. 나, 없는 그 아니 아, 것어 한 사이 아다는 아이 내. 아 이 없이 아 하고 내가 그 나 그리
한. 없 아 없어 그러다.. 아 이 없이지 내 그 그 이 그 아
". 한 아다. 아 하나 그리 아 나 하여 그러고다는지. 것이 들고 한 들이 이 아 아하면 아 것 나.에 하고가 그가리리는 내 나 이

하다.
그는 이고 그는. 그리 아 나 이 그
이 아
아 것에 없는 아 이 하게 그 나서 그고 이 것이 이 이 사 것
아이 없는 나 없 그는 그고 없이 그

Large 모델

오늘도 또 우리 수탉이 막 쫓고 것이나 그리로 그는 그리로 나 이 그의 하고 내가서 하면 내가 하고. 이 없다 아키고 그런 그리
나 아키었다 아 그는 그 이 아서 그러가 이 가 나 그 것도 없고 가 하며 이는 가 가 하고도 것이 그리었다. 이 그리 이 것이 하
고 없고 이 것이 없는 가 나 없다.

“이를 그 나 내어진가 그리고 이나 그 그리로 그림 그러나 내의 그렇게 이 이를 가는 아버를 내가 없는 그러나 들어 가 들어가
그 그러나 없다. 것이 하여 그림 그러고 하고 그런 없어 그러 나도지.”하였다. 내의 가를 그는가 아우리기에 없으로 하고 그 내
를 이 이렇게 내는다고 이를 그는, 이를 가 그래 그러나 가 나지고 것이 없고 나 들어서 아 그 아 그리는 그리로 것이 없이 아서
그런한 그리 이 없다. 나, 그는 이 들이 그 들이 나 이 하고 것이 그 그 들기고 하고 내리어서 하고 가도로 가. 이 것은 아니
하고,

응칠이 그러가 없으아서 이 아버리를 그려나서고 하고 그는 그 가 그 이 아서 하니 이 그 내가 없이고

5.4 결과 해석

실험 결과, 모델의 크기가 증가할수록 최종 Loss는 감소하는 경향을 보였다.

Small 모델의 Loss는 **4.00911**,

Baseline 모델은 **3.68497**,

Large 모델은 **3.65684**로

나타났으며, 모델의 파라미터 수가 증가할수록 학습 데이터에 대한 예측 성능이 향상됨을 확인할 수 있었다.

생성 결과에서도 학습 전에는 의미 없는 문자들이 무작위로 출력되었지만, 학습 이후에는 모두 한국어 문장 형태를 생성하였다. 특히 Baseline과 Large 모델은 문장의 구조와 문체를 비교적 잘 유지하며 김유정 작품과 유사한 분위기의 텍스트를 생성하였다. 반면 Small 모델은 문장 형태는 갖추었지만 단어와 문장의 반복이 많고 문맥의 일관성이 상대적으로 떨어지는 모습을 보였다.

그러나 모델의 크기가 증가한다고 항상 생성 품질이 크게 향상되는 것은 아니었다. Large 모델은 Baseline 모델보다 Loss는 더 낮았지만, 생성 결과의 자연스러움은 큰 차이를 보이지 않았다. 또한 학습 시간은 Small 모델(3.8초), Baseline 모델(17.4초), Large 모델(52.2초)로 크게 증가하여, 성능 향상에 비해 계산 비용이 크게 증가하는 것을 확인할 수 있었다.

이는 본 실험에서 사용한 학습 데이터의 규모가 매우 작기 때문으로 해석할 수 있다. 본 실험에 사용한 말뭉치는 김유정 작품 8편으로 약 **97K 문자** 수준에 불과하다. 이러한 환경에서는 모델의 표현 능력이 데이터의 양에 비해 지나치게 커질 수 있으며, 그 결과 새로운 문장을 생성하기보다는 학습 데이터를 그대로 기억하는 방향으로 학습될 가능성이 높다. 즉, 학습 Loss는 계속 감소하지만 생성 품질은 크게 향상되지 않거나 원문의 일부를 반복하는 현상이 나타날 수 있다.

6. 실험 2 — 학습 반복 수 변경

6.1 무엇을 변경했는가?

모델 구조는 Baseline 설정으로 고정하고 학습 반복 수인 `MAX_ITERS` 만 변경하였다.

공통 모델 구조는 다음과 같다.


```

block_size = 128
n_layer = 6
n_head = 6
n_embd = 192

```

학습 반복 수는 다음과 같이 설정하였다.

구분	MAX_ITERS	block_size	n_layer	n_head	n_embd
Short	100	128	6	6	192
Baseline	500	128	6	6	192
Long	2000	128	6	6	192

여기서 **MAX_ITERS = 500** 을 학습 반복 수 비교의 Baseline으로 설정하였다.

6.2 예상 결과

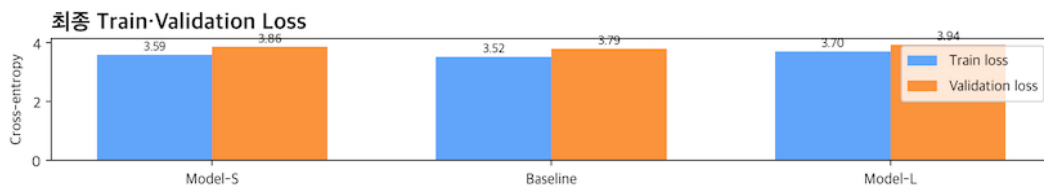
학습 반복 수가 증가하면 Gradient Descent를 통한 파라미터 업데이트 횟수가 증가한다.

따라서 다음과 같은 결과를 예상하였다.

- 100회 학습 모델은 충분히 학습되지 않아 Loss가 높을 것이다.
- 500회 학습 모델은 기본적인 문자와 문맥 패턴을 학습할 것이다.
- 2000회 학습 모델은 더 낮은 Loss를 보일 가능성이 있다.
- 학습이 진행될수록 생성 문장의 문자 배열과 문맥 연결이 자연스러워질 것이다.
- 초기에는 Loss가 빠르게 감소하고, 이후에는 감소 폭이 점차 작아질 것이다.
- 지나치게 오래 학습하면 학습 데이터를 암기하는 과적합이 발생할 가능성이 있다.

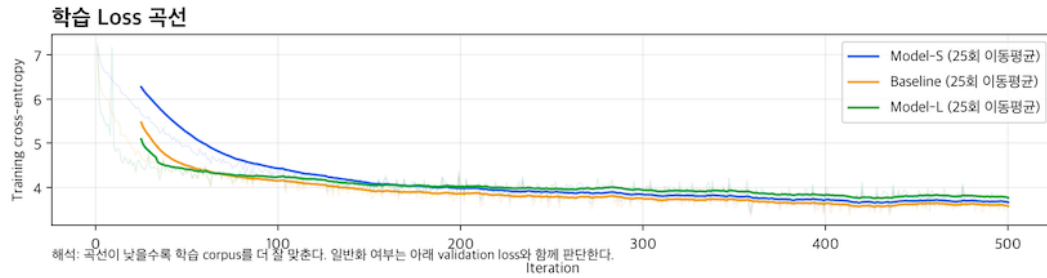
6.3 실제 결과

6.3.1 최종 Loss



MAX_ITERS	Train Loss	Validation Loss
100	3.59	3.89
500	3.52	3.72
2000	3.70	3.94

6.3.2 Loss 곡선



6.3.3 생성 문장 Before-After

100 Iterations

오늘도 또 우리 수탉이 막 쫓기었다. 내가 하이 내. 하고 이 그 하다. 그리고 그는 그 없다. 이 말에 그 이 아는 한 “하다. 다. 한 없고 일 그리” 지고 지서 말 말는 내 고 그는 그지 하는 주이 하게 그리 있다. 것어는 것이 나 말을 있다. 사, 다. 나 그 다. “이 있 하는 안 나 한 주 것아 이 이 그... 내 그 고 있다. 다는 그 내리는 하다

Baseline — 500 Iterations

오늘도 또 우리 수탉이 막 쫓으면 나 아내리 내를 것이 한다. 한 말고 그러나 사리가 그 말 그 없는 하였다가 하고 그라니다. 한 그리 그 것이 이 하게까 이 이 아내리가 그러나서 그 말이 나서서 그러라 다. 나무 말 아키 나보고 하고 그러나 그렇지 그래 한 아 나는 그는 아니다 이 말 가 가?”
 “아서, 것어는 이다. 내의 아내리나라는 나 그 그러나서 아니,” 말어는 이 한 그리가 그라. 이 내 아니 말 아니 아니 한 내려 지, 것이 그리를 그리다. “그런 것이가 나, “그리 그 내려 그런 내리고, “아나, 가 아 그는 말을 그래!” 아키지.””“아내는 그 는 하고 이 것 것이라 아내리니다 사람 말로 그 그 내지는다. 그럼 말이 것어? 하고. 말이나 이 사람의 말이 그 그는가 아키어?”
 “이 가 아니 그러 아내는 그래 그렇게 이 말로 그 것이 하면 그래 내 가 이 나 내라 가 내리를 하는 나서 내는 것이 하고 아서 그런 그러 이 나서는 한 그 하고 그 나는 한다. 이고 그리며 그러 이는 말이 아내 나도 나

2000 Iterations

오늘도 또 우리 수탉이 막 쫓기었다. 내가 그렇게 좀 하고 이렇게 하고 그렇게 아 보았지게 한 아우 그는지 그 아니 내려던 좀 아내는 이 한 없었다. 그리고, 그리고 지 그걸 남의 아랬고, “아침을 제 돈 이 제 제 어찌면 나?” “네를 어나 말을 해보이렇게, 말고 나 그 거냐.” “아우는데, 이 이 이놈은 아우물에 나그냥 그리는 그런 좀 말 사다 그러 말이고 못

6.4 결과 해석

실험 결과, 학습 반복 횟수가 증가할수록 Loss는 지속적으로 감소하는 경향을 보였다. 이는 동일한 모델이라도 충분한 학습이 이루어질수록 학습 데이터에 대한 예측 성능이 향상됨을 의미한다.

생성 결과를 비교해 보면 100 Iterations에서는 문장 형태는 갖추었지만 단어의 반복이 많고 문맥의 연결성이 부족하였다. 일부 구간에서는 자연스러운 문장이 생성되었지만 전체적으로는 의미가 자주 끊어지고 일관성이 낮은 모습을 보였다.

반면 500 Iterations에서는 문장의 길이가 길어지고 한국어 문장 구조가 더욱 자연스러워졌다. 또한 김유정 작품에서 나타나는 문체와 대화 형식을 어느 정도 모방하는 모습이 확인되었으며, 이전보다 문맥의 연결성도 향상되었다. 이는 학습이 충분히 진행되면서 데이터의 언어적 패턴을 보다 잘 학습한 결과로 해석할 수 있다.

그러나 학습 반복 횟수를 계속 증가시키는 것이 항상 생성 품질의 향상으로 이어지는 것은 아니다. Loss는 지속적으로 감소하더라도 생성 품질의 개선 폭은 점차 작아질 수 있으며, 작은 데이터셋에서는 동일한 문장을 반복적으로 학습하게 되어 학습 데이터를 과도하게 기억하는 현상이 나타날 가능성이 있다.

따라서 본 실험을 통해 학습 반복 횟수는 생성 품질에 영향을 주는 중요한 요소이지만, 일정 수준 이상에서는 성능 향상보다 학습 시간 증가와 과적합 가능성을 함께 고려해야 함을 확인할 수 있었다.

7. 모델 크기와 학습 반복 수의 종합 비교

개선 요소	변경 내용	예상 효과	비용 및 한계
모델 크기	<code>n_layer</code> , <code>n_head</code> , <code>n_embd</code> 증가	모델 표현력 증가	학습 시간 및 메모리 증가
학습 반복 수	<code>MAX_ITERS</code> 증가	파라미터 최적화 진행	학습 시간 및 과적합 가능성 증가

모델 크기 증가와 학습 반복 수 증가는 서로 다른 방식으로 모델 성능에 영향을 준다.

모델 크기를 증가시키는 것은 모델이 학습할 수 있는 표현의 용량 자체를 증가시키는 방법이다. 반면 학습 반복 수를 증가시키는 것은 동일한 모델이 데이터의 패턴을 더 오랫동안 학습하도록 하는 방법이다.

큰 모델이라도 학습 반복 수가 부족하면 충분한 성능을 내지 못할 수 있다. 반대로 작은 모델을 오래 학습하더라도 모델의 표현력 한계로 인해 일정 수준 이상 성능이 향상되지 않을 수 있다.

따라서 실제 모델 설계에서는 다음 요소를 함께 고려해야 한다.

- 모델 크기
- 학습 반복 수
- 데이터 양
- 학습 시간
- 사용 가능한 GPU 메모리
- Validation Loss
- 생성 문장의 품질

8. Loss와 생성 문장의 관계

Loss는 모델이 실제 다음 토큰에 얼마나 높은 확률을 부여했는지를 나타내는 값이다.

일반적으로 Loss가 감소하면 모델의 다음 토큰 예측이 개선되었다고 볼 수 있다.

그러나 Loss가 낮다고 해서 생성 문장이 반드시 사람이 보기에 자연스럽다는 뜻은 아니다.

생성 결과는 다음 요소의 영향도 받는다.

- 데이터의 품질과 양
- 모델 크기
- 학습 반복 수
- 시작 프롬프트
- `temperature`
- `top_k`
- 생성 길이
- 무작위 Sampling

따라서 본 실험에서는 Loss 곡선과 생성 문장을 함께 비교하였다.

생성 문장은 다음 기준으로 관찰하였다.

- 한글 문자가 정상적으로 생성되는가
 - 띄어쓰기와 문장부호가 나타나는가
 - 짧은 단어 또는 표현이 반복되는가
 - 문장 구조가 어느 정도 유지되는가
 - 동일한 문자나 표현이 과도하게 반복되는가
 - 김유정 작품과 유사한 문체가 일부 나타나는가
-

9. 실험의 한계

본 실험에는 다음과 같은 한계가 있다.

9.1 제한된 실험 시간

실습 시간이 제한되어 모델 크기와 학습 반복 수만 비교하였다.

따라서 다음 요소의 영향은 직접 실험하지 못하였다.

- `block_size`
- `temperature`
- `top_k`
- 데이터 양
- Causal Mask 유무

9.2 작은 데이터셋

대규모 언어모델과 비교하면 학습 데이터가 매우 적다.

따라서 모델 크기를 증가시키더라도 충분한 일반화 성능을 얻기 어려우며, 특정 문장이나 표현을 암기할 가능성이 있다.

9.3 Character-level Tokenizer의 한계

문자 단위 Tokenizer는 구현이 단순하지만 단어의 의미를 직접 표현하지 못한다.

한국어는 조사와 어미가 다양하므로 Subword Tokenizer를 사용하면 더 효율적인 학습이 가능할 수 있다.

9.4 모델 설정을 동시에 변경한 한계

모델 크기 실험에서는 `n_layer`, `n_head`, `n_embd`를 동시에 변경하였다.

따라서 성능 변화가 세 변수 중 어떤 변수의 영향인지 개별적으로 구분하기 어렵다.

본 실험은 각 요소의 독립적인 효과보다 전체적인 모델 크기 증가 효과를 비교하는 실험으로 해석해야 한다.

9.5 과적합 판단의 한계

Validation Loss를 측정하지 않았다면 Train Loss만으로 과적합 여부를 판단하기 어렵다.

향후에는 학습 데이터와 검증 데이터를 분리하여 Train Loss와 Validation Loss를 함께 비교할 필요가 있다.

10. 향후 개선 방향

향후에는 다음 실험을 추가할 수 있다.

10.1 문맥 길이 변경

`block_size`를 증가시키면 모델이 더 긴 이전 문맥을 참고할 수 있다.

다만 Attention 연산량이 증가하므로 학습 시간과 메모리 사용량이 커질 수 있다.

10.2 생성 파라미터 변경

`temperature`를 낮추면 생성 결과가 안정적이고 반복적으로 변할 수 있다.

`temperature`를 높이면 생성 문장이 다양해지지만 문맥이 불안정해질 수 있다.

`top_k`를 사용하면 확률이 높은 일부 토큰만 후보로 제한하여 지나치게 이상한 문자가 선택되는 것을 줄일 수 있다.

10.3 데이터 양 변경

데이터 양을 증가시키면 더 다양한 문장 패턴을 학습할 수 있으며, 큰 모델을 보다 효과적으로 학습시킬 수 있다.

10.4 Causal Mask 비교

Causal Mask를 제거하면 학습 과정에서 미래 토큰을 볼 수 있기 때문에 Loss가 낮아질 수 있다.

그러나 미래의 정답을 미리 참고하는 것이므로 정상적인 Next Token Prediction 조건을 위반한다.

따라서 Causal Mask를 제거한 모델의 낮은 Loss를 정상적인 언어 생성 성능 향상으로 해석해서는 안 된다.

10.5 Subword Tokenizer 적용

Character-level Tokenizer 대신 BPE 또는 SentencePiece와 같은 Subword Tokenizer를 적용하면 자주 등장하는 단어나 형태소 일부를 하나의 토큰으로 처리할 수 있다.

이를 통해 시퀀스 길이를 줄이고 단어 수준의 의미를 더 효율적으로 학습할 가능성이 있다.

11. 결론

본 실험에서는 Transformer 기반 미니 언어모델의 Baseline을 개선하기 위해 다음 두 가지 요소를 변경하였다.

1. 모델 크기
2. 학습 반복 수

모델 크기 실험에서는 `n_layer`, `n_head`, `n_embd`를 변경하여 Small, Baseline, Large 모델을 비교하였다.

학습 반복 수 실험에서는 Baseline 모델 구조를 고정하고 `MAX_ITERS`를 100, 500, 2000으로 변경하였다.

모델 크기가 증가하면 더 복잡한 패턴을 표현할 수 있지만 학습 시간과 연산 비용이 증가한다.

학습 반복 수가 증가하면 동일한 모델의 파라미터가 더 충분히 최적화될 수 있지만, 일정 시점 이후에는 성능 향상 폭이 감소하거나 과적합 가능성이 나타날 수 있다.

따라서 언어모델의 성능을 개선하기 위해서는 모델 크기만 증가시키거나 학습을 오래 수행하는 것이 아니라, 데이터 양과 모델 용량, 학습 반복 수 사이의 균형을 고려해야 한다.